

Tutorial 4: Computer Exercises (Chapter 2) - C6, C7, C8

Wenhui Xu

ECO375

Contents

Overview	2
0.5 A quick demonstration: the pipe operator %>%	2
The basic rule	2
A two-step pipeline	3
The placeholder . (when x is not the first argument)	3
A common pattern in data analysis	4
C6: MEAP93 (math10 vs log(expend))	4
What the problem is asking (plain English)	4
Data and a quick look	4
(i) Diminishing effect intuition	5
(ii) Why b1/10 corresponds to a 10% increase in expend	5
(iii) Estimate the model and report the equation	5
(iv) Effect of a 10% increase in spending	6
(v) Can fitted values exceed 100?	6
C7: CHARITY (gift vs mailsyear)	7
What the problem is asking (plain English)	7
Data checks	7
(i) Average gift and percent zero	7
(ii) Mailings per year: mean, min, max	7
(iii) Estimate the regression and report	8
(iv) Interpret b1 and net gain per mailing	9
(v) Smallest predicted gift and can we predict zero?	9

C8: Simulation and OLS properties	9
What the problem is asking (plain English)	9
A clean, reproducible implementation	9
(ii) Why is the sample mean of u not exactly zero?	10
(iv) OLS residual properties (sample analog of equation 2.60)	10
(v) Same computation using the <i>true</i> errors u	11
(vi) Repeat (i)-(iii) with a new sample	11
Wrap-up	11

Overview

This tutorial covers three computer exercises from Wooldridge (2019), Chapter 2:

- **C6 (MEAP93):** a level-log model of math pass rates on spending per student.
- **C7 (CHARITY):** a simple regression of charitable giving on mailings per year.
- **C8 (Simulation):** generating data to illustrate sampling variation and OLS residual properties.

We will use base R as much as possible.

```
# Install/load packages only if needed
if (!requireNamespace("wooldridge", quietly = TRUE)) install.packages("wooldridge")
library(wooldridge)

# Optional helpers (not strictly required)
if (!requireNamespace("dplyr", quietly = TRUE)) install.packages("dplyr")
library(dplyr)
```

0.5 A quick demonstration: the pipe operator %>%

In this course we will often use the **pipe operator** %>% (from `dplyr` / `magrittr`). Read it as “**then**”: it takes the object on the left and passes it into the function on the right.

The basic rule

```
# These two lines do the same thing:
sqrt(16)
```

```
## [1] 4
```

```
16 %>% sqrt()
```

```
## [1] 4
```

In general:

- $x \%>\% f()$ means $f(x)$
- $x \%>\% f(a = 1)$ means $f(x, a = 1)$

A two-step pipeline

```
demo_df <- data.frame(  
  math10 = c(20, 40, 60),  
  expend = c(3500, 4200, 5100)  
)
```

```
# Without pipes (read inside-out):  
summary(dplyr::select(demo_df, math10, expend))
```

```
##      math10      expend  
## Min.    :20   Min.    :3500  
## 1st Qu.:30   1st Qu.:3850  
## Median :40   Median :4200  
## Mean   :40   Mean    :4267  
## 3rd Qu.:50   3rd Qu.:4650  
## Max.   :60   Max.    :5100
```

```
# With pipes (read top-down):  
demo_df %>%  
  dplyr::select(math10, expend) %>%  
  summary()
```

```
##      math10      expend  
## Min.    :20   Min.    :3500  
## 1st Qu.:30   1st Qu.:3850  
## Median :40   Median :4200  
## Mean   :40   Mean    :4267  
## 3rd Qu.:50   3rd Qu.:4650  
## Max.   :60   Max.    :5100
```

The placeholder . (when x is not the first argument)

Sometimes the function you want to call does *not* take your object as the first input. Then you can use `.` to mark where the left-hand object should go.

```
v <- c(1, 2, 3, 4, 5)  
  
# Same result:  
paste(v, collapse = "-")
```

```
## [1] "1-2-3-4-5"
```

```
v %>% paste(collapse = "-")
```

```
## [1] "1-2-3-4-5"
```

```
s <- "1-2-3-4-5"  
  
# gsub(pattern, replacement, x)  
gsub("-", "_", s)
```

```
## [1] "1_2_3_4_5"
```

```
# Using the dot placeholder:  
s %>% gsub("-", "_", .)
```

```
## [1] "1_2_3_4_5"
```

A common pattern in data analysis

“Take a dataset -> select columns -> create new variables -> summarize”

```
demo_df %>%  
  dplyr::select(math10, expend) %>%  
  dplyr::mutate(ln_expend = log(expend)) %>%  
  summary()
```

```
##      math10      expend      ln_expend  
## Min.   :20    Min.   :3500    Min.   :8.161  
## 1st Qu.:30    1st Qu.:3850    1st Qu.:8.252  
## Median :40    Median :4200    Median :8.343  
## Mean   :40    Mean   :4267    Mean   :8.347  
## 3rd Qu.:50    3rd Qu.:4650    3rd Qu.:8.440  
## Max.   :60    Max.   :5100    Max.   :8.537
```

Note: %>% works because we load `dplyr` in this tutorial. If you ever see an error like “could not find function %>%”, it means the package was not loaded.

C6: MEAP93 (math10 vs log(expend))

What the problem is asking (plain English)

1. Conceptually: does spending have diminishing returns for pass rates?
2. Show why, in the model $\text{math10} = b_0 + b_1 \log(\text{expend}) + u$, the quantity $b_1/10$ is the change in **percentage points** in math10 when expend rises by 10%.
3. Estimate the regression and report the fitted equation with n and R^2 .
4. Use the estimate to compute the impact of a 10% spending increase.
5. Check whether fitted values exceed 100, and explain why this is not a major concern here.

Data and a quick look

```
data("meap93")  
meap93 <- as_tibble(meap93)  
  
# Key variables  
meap93 %>% select(math10, expend) %>% summary()
```

```
##      math10      expend  
## Min.   : 1.90    Min.   :3332  
## 1st Qu.:16.62    1st Qu.:3821  
## Median :23.40    Median :4145  
## Mean   :24.11    Mean   :4377  
## 3rd Qu.:30.05    3rd Qu.:4659  
## Max.   :66.70    Max.   :7419
```

(i) Diminishing effect intuition

A linear-in-levels model assumes each extra dollar has the same marginal effect. A log specification allows the effect of spending to shrink as spending gets larger.

Here we proceed with the **level-log** model:

$$\text{math10} = b_0 + b_1 \log(\text{expend}) + u.$$

(ii) Why $b_1/10$ corresponds to a 10% increase in expend

A 10% increase means $\text{expend_new} = 1.10 * \text{expend}$. Then:

$$\log(\text{expend_new}) - \log(\text{expend}) = \log(1.10) \approx 0.0953.$$

So the change in predicted math10 is approximately:

$$b_1 * \log(1.10) \approx b_1 * 0.10 = b_1/10.$$

Because math10 is measured in **percentage points**, this change is in percentage points too.

(iii) Estimate the model and report the equation

```
c6_fit <- lm(math10 ~ log(expend), data = meap93)
summary(c6_fit)
```

```
##
## Call:
## lm(formula = math10 ~ log(expend), data = meap93)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.343  -7.100  -0.914   6.148  39.093
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -69.341     26.530  -2.614  0.009290 **
## log(expend)   11.164       3.169   3.523  0.000475 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.35 on 406 degrees of freedom
## Multiple R-squared:  0.02966,    Adjusted R-squared:  0.02727
## F-statistic: 12.41 on 1 and 406 DF,  p-value: 0.0004752
```

```
b0_hat <- coef(c6_fit)[1]
b1_hat <- coef(c6_fit)[2]
n_c6    <- nobs(c6_fit)
r2_c6   <- summary(c6_fit)$r.squared
```

```
list(b0_hat = b0_hat, b1_hat = b1_hat, n = n_c6, r2 = r2_c6)
```

```
## $b0_hat
## (Intercept)
##    -69.34116
##
```

```
## $b1_hat
## log(expend)
##      11.1644
##
## $n
## [1] 408
##
## $r2
## [1] 0.02966334
```

Write the fitted equation:

```
cat(sprintf("Estimated equation:\n\math10 = %.3f + %.3f * log(expend)\n\n = %d,    R^2 = %.4f\n",
            b0_hat, b1_hat, n_c6, r2_c6))
```

Estimated equation:

$\text{math10} = -69.341 + 11.164 * \log(\text{expend})$

$n = 408, R^2 = 0.0297$

(iv) Effect of a 10% increase in spending

```
effect_10pct_exact <- b1_hat * log(1.10)
effect_10pct_approx <- b1_hat / 10

c(effect_10pct_exact = effect_10pct_exact,
  effect_10pct_approx = effect_10pct_approx)
```

```
## effect_10pct_exact.log(expend) effect_10pct_approx.log(expend)
##                1.064081                1.116440
```

Interpretation: a 10% increase in spending is associated with an increase of about the number above (in **percentage points**) in the math pass rate.

(v) Can fitted values exceed 100?

```
meap93 <- meap93 %>% mutate(math10_hat = fitted(c6_fit))

summary(meap93$math10_hat)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  21.22   22.75   23.65   24.11   24.96   30.15
```

```
sum(meap93$math10_hat > 100)
```

```
## [1] 0
```

```
max(meap93$math10_hat)
```

```
## [1] 30.15376
```

Why this is not much of a worry here:

- The sample range of $\log(\text{expend})$ is limited, so predicted values tend to stay in a realistic range.
- Even if a few fitted values were slightly above 100, that is a limitation of the linear regression functional form, not necessarily a data problem.

C7: CHARITY (gift vs mailsyear)

What the problem is asking (plain English)

1. Compute average gift and the percentage of people who gave zero.
2. Summarize mailings per year: mean, min, max.
3. Estimate $\text{gift} = b_0 + b_1 \text{ mailsyear} + u$ and report n and R^2 .
4. Interpret b_1 and discuss net gain if each mailing costs one guilder.
5. Find the smallest predicted gift. Can the model ever predict exactly zero?

Data checks

```
data("charity")
charity <- as_tibble(charity)

charity %>% select(gift, mailsyear) %>% summary()
```

```
##      gift      mailsyear
## Min.   : 0.000   Min.   :0.25
## 1st Qu.: 0.000   1st Qu.:1.75
## Median : 0.000   Median :2.00
## Mean   : 7.444   Mean   :2.05
## 3rd Qu.:10.000   3rd Qu.:2.50
## Max.   :250.000   Max.   :3.50
```

(i) Average gift and percent zero

```
avg_gift <- mean(charity$gift, na.rm = TRUE)
pct_zero <- mean(charity$gift == 0, na.rm = TRUE) * 100

c(avg_gift = avg_gift, pct_zero = pct_zero)
```

```
## avg_gift pct_zero
## 7.44447 60.00469
```

(ii) Mailings per year: mean, min, max

```
c(mean_mailsyear = mean(charity$mailsyear, na.rm = TRUE),
  min_mailsyear = min(charity$mailsyear, na.rm = TRUE),
  max_mailsyear = max(charity$mailsyear, na.rm = TRUE))
```

```
## mean_mailsyear min_mailsyear max_mailsyear
## 2.049555 0.250000 3.500000
```

(iii) Estimate the regression and report

```
c7_fit <- lm(gift ~ mailsyear, data = charity)
summary(c7_fit)
```

```
##
## Call:
## lm(formula = gift ~ mailsyear, data = charity)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.287  -7.976  -5.976   2.687  245.999
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.0141     0.7395   2.724  0.00648 **
## mailsyear     2.6495     0.3431   7.723  1.4e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 14.96 on 4266 degrees of freedom
## Multiple R-squared:  0.01379,    Adjusted R-squared:  0.01356
## F-statistic: 59.65 on 1 and 4266 DF,  p-value: 1.404e-14
```

```
b0_c7 <- coef(c7_fit)[1]
b1_c7 <- coef(c7_fit)[2]
n_c7  <- nobs(c7_fit)
r2_c7 <- summary(c7_fit)$r.squared

list(b0_hat = b0_c7, b1_hat = b1_c7, n = n_c7, r2 = r2_c7)
```

```
## $b0_hat
## (Intercept)
##      2.01408
##
## $b1_hat
## mailsyear
##      2.649546
##
## $n
## [1] 4268
##
## $r2
## [1] 0.01378962
```

```
cat(sprintf("Estimated equation:\n\ngift = %.3f + %.3f * mailsyear\n\nn = %d,    R^2 = %.4f\n",
            b0_c7, b1_c7, n_c7, r2_c7))
```

Estimated equation:

gift = 2.014 + 2.650 * mailsyear

n = 4268, R² = 0.0138

(iv) Interpret b1 and net gain per mailing

- b1 is the **expected change in gift (guilders)** when mailings per year increases by 1.
- If a mailing costs 1 guilder, then the expected net gain from one additional mailing is approximately (b1 - 1).

```
expected_net_gain_per_mailing <- b1_c7 - 1
expected_net_gain_per_mailing
```

```
## mailsyear
## 1.649546
```

Important nuance: this is an **average** effect across the sample, not a guarantee that every mailing to every person produces a gain.

(v) Smallest predicted gift and can we predict zero?

```
charity <- charity %>% mutate(gift_hat = fitted(c7_fit))

min_pred <- min(charity$gift_hat, na.rm = TRUE)
min_pred
```

```
## [1] 2.676466
```

Can the simple regression ever predict zero?

- The fitted value is: $\text{gift_hat} = b0_hat + b1_hat * \text{mailsyear}$.
- This equals zero only if $\text{mailsyear} = -b0_hat / b1_hat$, which may not be an integer and may be outside the observed range.
- If $b0_hat > 0$ and mailsyear is nonnegative, then predictions will tend to be positive.

C8: Simulation and OLS properties

What the problem is asking (plain English)

We simulate data from a known population process:

1. Draw x from $\text{Uniform}(0, 10)$ and compute sample mean and sd.
2. Draw u from $\text{Normal}(0, 36)$ and compute sample mean and sd; explain why sample mean is not exactly zero.
3. Generate $y = 1 + 2x + u$, run OLS, and compare estimates to the true values (1 and 2).
4. Compute residuals and verify the OLS residual properties: $\sum(uhat) = 0$ and $\sum(x*uhat) = 0$ (up to rounding).
5. Replace residuals by the *true* errors u and see these properties do not hold in finite samples.
6. Repeat (i)-(iii) with a new sample and explain why estimates change.

A clean, reproducible implementation

```

set.seed(375) # reproducibility

n <- 500

# (i)  $x \sim \text{Uniform}(0,10)$ 
x <- runif(n, min = 0, max = 10)
mean_x <- mean(x)
sd_x <- sd(x)

# (ii)  $u \sim \text{Normal}(0,36)$  so  $sd = 6$ 
u <- rnorm(n, mean = 0, sd = 6)
mean_u <- mean(u)
sd_u <- sd(u)

# (iii)  $y = 1 + 2x + u$ 
y <- 1 + 2*x + u

fit <- lm(y ~ x)

b0_hat <- coef(fit)[1]
b1_hat <- coef(fit)[2]

list(mean_x = mean_x, sd_x = sd_x,
      mean_u = mean_u, sd_u = sd_u,
      b0_hat = b0_hat, b1_hat = b1_hat)

```

```

## $mean_x
## [1] 4.908538
##
## $sd_x
## [1] 2.926813
##
## $mean_u
## [1] -0.01735551
##
## $sd_u
## [1] 6.233669
##
## $b0_hat
## (Intercept)
## 0.6658578
##
## $b1_hat
## x
## 2.064538

```

(ii) Why is the sample mean of u not exactly zero?

Even though $E[u] = 0$ in the population, a finite random sample will almost never have sample mean exactly 0. As n grows, the sample mean tends to get closer to 0 (law of large numbers).

(iv) OLS residual properties (sample analog of equation 2.60)

```

uhat <- resid(fit)

# Theoretical properties (in sample, with intercept):
# sum(uhat) = 0 and sum(x * uhat) = 0, up to rounding error.
c(sum_uhat = sum(uhat),
  sum_x_uhat = sum(x * uhat))

##          sum_uhat    sum_x_uhat
## 2.131628e-14 -4.973799e-13

```

(v) Same computation using the *true* errors u

```

c(sum_u = sum(u),
  sum_x_u = sum(x * u))

##          sum_u    sum_x_u
## -8.677757 233.275426

```

Conclusion: the residual properties are guaranteed by the OLS algebra, but the same equalities do not generally hold if we plug in the unobserved true errors u .

(vi) Repeat (i)-(iii) with a new sample

```

set.seed(376) # new seed -> new sample

x2 <- runif(n, 0, 10)
u2 <- rnorm(n, 0, 6)
y2 <- 1 + 2*x2 + u2

fit2 <- lm(y2 ~ x2)
coef(fit2)

```

```

## (Intercept)          x2
##    1.631173    1.967550

```

Why are the new estimates different?

Because each sample has different random draws of x and u , so the OLS estimates vary from sample to sample. With larger n , they concentrate around the true values (consistency).

Wrap-up

- C6 shows how to interpret a level-log model and translate log changes into percentage changes.
- C7 shows how to interpret a simple regression, including the difference between an average effect and an individual outcome.
- C8 connects probability theory to the mechanics of OLS: residual properties hold by construction, and sampling variation is unavoidable.